Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07 Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 2

Дисциплина(модуль)/предмет МДК 02.02. Инструментальные средства разработки и отладки программного обеспечения

Тема: Проектирование уровней интеграции и схемы взаимодействия модулей для учебного проекта

Студент (ка)/учащийся: Иванов Егор Михайлович / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: Чудинова А.А. / _____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «10» октября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ОБОСНОВАНИЕ ВЫБОРА УРОВНЕЙ ИНТЕГРАЦИИ.....4
2 ЗАДАНИЕ № 1. РАЗРАБОТКА МОДУЛЬНОЙ СТРУКТУРЫ ПРОЕКТА…6
2.1 Выбранный проект и границы системы.....6
2.2 Состав компонентов системы.....6
2.3 Схема модульной структуры.....6
2.4 Описание схемы взаимодействия.....8
3 ЗАДАНИЕ № 2. РАЗРАБОТКА ПЕРЕЧНЯ АРТЕФАКТОВ И ПРОТОКОЛОВ ПРОЕКТА.....11
ЗАКЛЮЧЕНИЕ.....13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....14

ВВЕДЕНИЕ

Цель работы — на основе технического описания мини-проекта «Система агрегации и интеграции данных интернет-магазина» обосновать выбор уровней интеграции для конкретных модулей системы, разработать модульную структуру проекта и составить спецификацию артефактов и протоколов взаимодействия между источниками и приёмниками данных.

Задачи работы:

– обосновать выбор уровня интеграции (данных, интерфейсов или бизнес-логики) для каждого модуля разрабатываемой системы;

– определить границы и состав компонентов системы и разделить их на модуль источника/интерфейса, модуль бизнес-логики и модуль приёмника/хранилища;

– построить схему модульной структуры проекта (диаграмму связей) и описать по шагам, как компоненты передают друг другу управление и данные;

– составить таблицу сопоставления объектов данных и протоколов взаимодействия между источниками и приёмниками.

Исходные данные: техническое описание мини-проекта [1] и результаты практической работы № 1, в которой определены источники И1, И2 и приёмники П1, П2, а также спроектирована структура репозитория `shop-integration`. Схема модулей и спецификация, разработанные в данной работе, размещены в каталоге `docs/pr2` репозитория проекта.

1 ОБОСНОВАНИЕ ВЫБОРА УРОВНЕЙ ИНТЕГРАЦИИ

Техническое описание [1] предусматривает три уровня интеграции: уровень данных (СУБД и структурированные файлы), уровень интерфейсов (REST API внешней торговой площадки) и уровень бизнес-логики (валидация заказов и расчёт остатков). В отличие от практической работы № 1, где уровни выбирались для системы в целом, здесь уровень определяется для каждого модуля отдельно — по тому, как именно модуль получает или отдаёт данные и какой доступ к смежной системе у него есть.

Таблица 1 – Выбор уровня интеграции для модулей системы

Модуль (компонент)	**Уровень интеграции**	**Обоснование выбора**
connectors/sftp_client — загрузка файла выгрузки склада (И1)	Уровень данных	Складская система не имеет API и отдаёт остатки и цены только пакетным файлом CSV/XML по SFTP. Интеграция возможна лишь через обмен файлами, то есть на уровне данных.
webhooks/receiver — приём новых заказов (И2)	Уровень интерфейсов (API)	Интернет-магазин сам отправляет события через вебхуки (HTTPS, JSON). Модуль публикует Web API — точку приёма POST-запросов; это даёт обработку заказа в режиме, близком к реальному времени.
parsers — разбор CSV, XML, JSON	Уровень данных	Компонент не вызывает внешние системы, а приводит разные форматы к единым внутренним моделям; это преодоление синтаксических различий структур данных.
services/order_validator, stock_calculator, order_status_sync, catalog_sync	Уровень бизнес-логики	Здесь реализуются правила предприятия: заказ записывается только после проверки, на площадку уходит рассчитанный остаток (склад минус резерв), а не «сырые» цифры. Эти правила нельзя переложить ни на СУБД, ни на API площадки.
connectors/db_repository — запись заказов, чтение отгрузок (П2)	Уровень данных	Учётная система принимает новые заказы прямой записью в свою базу данных; доступ — через SQL-запросы по протоколу СУБД поверх TCP.
connectors/ marketplace_client — передача остатков, статусов, каталога (П1)	Уровень интерфейсов (API)	Маркетплейс не даёт доступа к своей БД; обмен возможен только через опубликованный REST API по HTTPS в формате JSON.

Итог: система использует все три уровня, но каждый модуль работает на одном, «своём» уровне. Это упрощает сопровождение: изменение формата файла склада затрагивает только парсер, изменение API маркетплейса —

только `marketplace_client`, а изменение правил расчёта остатков — только сервисный слой. Ведущим, как и в практической работе № 1, остаётся уровень интерфейсов, так как через него проходят события о заказах и все данные для внешней площадки.

2 ЗАДАНИЕ № 1. РАЗРАБОТКА МОДУЛЬНОЙ СТРУКТУРЫ ПРОЕКТА

2.1 Выбранный проект и границы системы

Выбранный проект / кейс: мини-проект «Система агрегации и интеграции данных интернет-магазина» (репозиторий `shop-integration`).

Границы системы: в состав разрабатываемого модуля интеграции входит только код каталога `src/` с конфигурацией `config/`. Складская система, интернет-магазин, маркетплейс и база данных учётной системы являются внешними системами: модуль не изменяет их код, а взаимодействует с ними через файлы, API и SQL-запросы.

2.2 Состав компонентов системы

Таблица 2 – Границы и состав компонентов системы

Модуль	Компоненты (каталоги src/)	Назначение
Модуль 1 (Источник / Интерфейс): модуль приёма данных и Web API	connectors/sftp_client webhooks/receiver parsers/	Получает данные из источников И1 и И2: забирает файл выгрузки склада по SFTP, принимает вебхуки о заказах (POST /webhooks/orders) с проверкой подписи HMAC, разбирает CSV, XML и JSON во внутренние модели.
Модуль 2 (Бизнес-логика / Обработка): модуль валидации и расчёта остатков	models/ services/order_validator services/stock_calculator services/ order_status_sync services/catalog_sync	Проверяет заказы (обязательные поля, существование товара, достаточность остатка), рассчитывает актуальные остатки с учётом резерва, готовит статусы и трек-номера, синхронизирует каталог.
Модуль 3 (Приёмник / Хранилище): модуль записи в БД и выгрузки на площадку	connectors/db_repository connectors/ marketplace_client	Записывает проверенные заказы в БД учётной системы (П2), читает из неё отгруженные заказы; отправляет остатки, статусы, трек-номера и карточки товаров на сервер маркетплейса (П1).
Вспомогательные компоненты	utils/ config/	Журналирование, повтор запросов при сбоях сети, параметры подключений (секреты — только в .env).

2.3 Схема модульной структуры

Схема связей модулей приведена на рисунке 1. Внешние системы изображены условными обозначениями ГОСТ 19.701-90 [4]: файл выгрузки склада — символом «документ», обмен через сеть — символом «данные»,

база данных учётной системы — символом «запоминающее устройство с прямым доступом». Компоненты модуля показаны прямоугольниками, границы модулей — сплошными рамками, граница всего модуля интеграции — штрихпунктирной рамкой. Связи проведены только горизонтальными и вертикальными линиями; штриховая линия обозначает внутренний сигнал о резерве товара. Подписи на линиях обозначают поток (А — остатки и цены, Б — заказы, В — статусы и трек-номера, Г — каталог) и номер шага в потоке.

Схема строится скриптом `docs/pr2/modules_diagram.py` с помощью библиотеки Matplotlib [5], поэтому хранится в репозитории как код, и её изменения видны в истории коммитов.

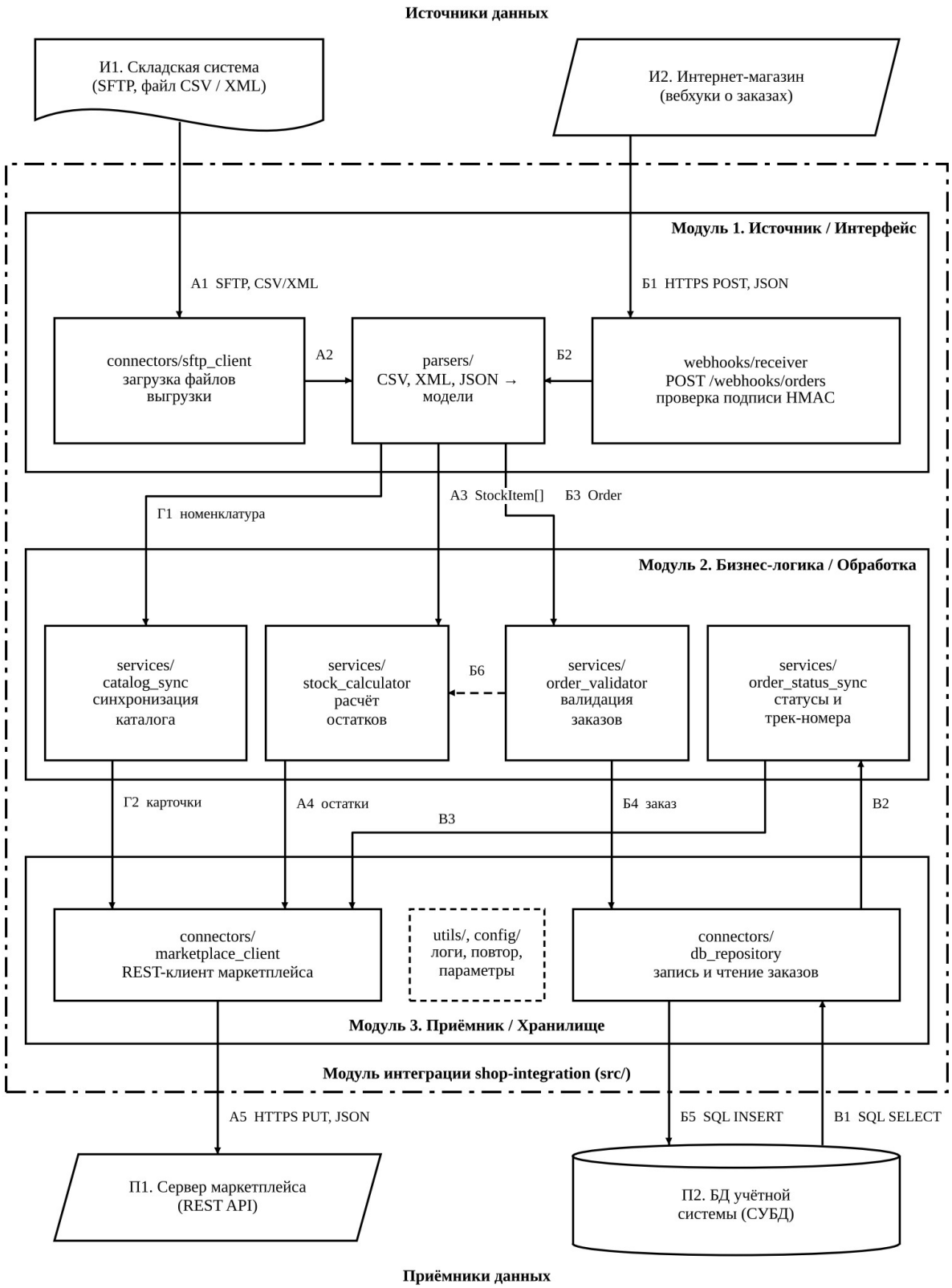


Рисунок 1 – Схема модульной структуры модуля интеграции

2.4 Описание схемы взаимодействия

Поток А «Остатки и цены» (запускается по расписанию скриптом из каталога `scripts/`):

1) А1. `sftp_client` подключается к SFTP-серверу складской системы и скачивает свежий файл выгрузки (CSV или XML).

2) А2. Файл передаётся в `parsers`, который проверяет структуру и преобразует строки в список объектов StockItem (артикул, количество, цена).

3) А3. Список передаётся в `stock_calculator`, который вычитает из складского количества резерв по принятым, но ещё не отгруженным заказам и не допускает отрицательных значений.

4) А4–А5. Рассчитанные остатки передаются в `marketplace_client`, который отправляет их на маркетплейс запросом HTTPS PUT в формате JSON; при сбое сети запрос повторяется (`utils`).

Поток Б «Заказы» (запускается событием — вебхуком):

1) Б1. Интернет-магазин отправляет POST-запрос с JSON о новом заказе на адрес `/webhooks/orders`; `webhooks/receiver` проверяет подпись запроса и сразу отвечает кодом 200, чтобы магазин не повторял отправку.

2) Б2–Б3. Тело запроса разбирается парсером в объект Order и передаётся в `order_validator`.

3) Б4–Б5. Проверенный заказ передаётся в `db_repository` и записывается в БД учётной системы SQL-запросом INSERT в одной транзакции; заказ с ошибкой не записывается, а попадает в журнал.

4) Б6. После записи заказа валидатор сообщает `stock_calculator` о резерве, и остатки на площадке уменьшаются при следующей передаче.

Поток В «Статусы и трек-номера»: `db_repository` выбирает из БД учётной системы заказы, отгруженные со времени последней синхронизации (В1); `order_status_sync` формирует для них статусы и трек-номера (В2); `marketplace_client` передаёт их на маркетплейс (В3). Как и в практической работе № 1, принято допущение, что трек-номер формируется в учётной системе при отгрузке.

Поток Г «Каталог»: номенклатура из файла выгрузки (после разбора парсером) передаётся в `catalog_sync` (Г1), который сравнивает её с уже

выгруженной и отправляет на маркетплейс только новые и изменённые карточки товаров в формате JSON (Г2).

Управление во всех потоках передаётся сверху вниз: модуль 1 вызывает модуль 2, модуль 2 — модуль 3. Обратных вызовов между модулями нет, поэтому каждый модуль можно тестировать отдельно, подменяя соседний модуль заглушкой (тестовые данные — в `tests/fixtures`).

3 ЗАДАНИЕ № 2. РАЗРАБОТКА ПЕРЕЧНЯ АРТЕФАКТОВ И ПРОТОКОЛОВ ПРОЕКТА

Спецификация артефактов и протоколов взаимодействия приведена в таблице 3. Для каждой связи указаны компоненты, уровень интеграции, транспортный протокол, формат данных и назначение.

Таблица 3 – Спецификация артефактов и протоколов взаимодействия

Компонент-источник	Компонент-приёмник	Уровень интеграции	Транспортный протокол	Формат данных / артефакт	Назначение / передаваемые данные
Складская система (И1)	Модуль 1: sftp_client	Данные	SFTP (поверх SSH, TCP 22)	CSV / XML, файл stock_ГГГГММДД.csv	Остатки товаров и цены (пакетно, по расписанию)
Внешняя витрина — интернет-магазин (И2)	Модуль 1: webhooks/receiver	Интерфейсы (API)	HTTPS (Webhooks, POST)	JSON	Данные о новом заказе: состав, покупатель, доставка
Модуль 1: webhooks/receiver	Интернет-магазин (И2)	Интерфейсы (API)	HTTPS (ответ на запрос)	Код ответа HTTP 200 / 400 / 401	Подтверждение приёма вебхука или причина отказа
Модуль 1: parsers	Модуль 2: services	Бизнес-логика	Вызов внутри процесса (без сети)	Объекты моделей StockItem, Order, Product	Разобранные данные в едином внутреннем формате
Модуль 2: order_validator	Модуль 3: db_repository → БД учётной системы (П2)	Данные	TCP (протокол СУБД), SQL	Записи таблиц orders, order_items	Запись проверенного заказа (транзакция)
БД учётной системы (П2)	Модуль 3: db_repository → order_status_sync	Данные	TCP (протокол СУБД), SQL	Результат SQL SELECT	Отгруженные заказы, статусы, трек-номера
Модуль 2: stock_calculator	Модуль 3: marketplace_client → маркетплейс (П1)	Интерфейсы (API)	HTTPS (REST, PUT)	JSON	Рассчитанные остатки товаров
Модуль 2: order_status_sync	Модуль 3: marketplace_client → маркетплейс (П1)	Интерфейсы (API)	HTTPS (REST, POST)	JSON	Статусы заказов и трек-номера отправлений
Модуль 2: catalog_sync	Модуль 3: marketplace_client → маркетплейс (П1)	Интерфейсы (API)	HTTPS (REST, POST/PUT)	JSON (карточки товаров)	Новые и изменённые позиции каталога
Маркетплейс (П1)	Модуль 3: marketplace_client	Интерфейсы (API)	HTTPS (ответ на запрос)	JSON, коды HTTP 2xx / 4xx / 5xx	Подтверждение или ошибка; при 5xx — повтор запроса
config/ и файл .env	Все модули	— (конфигурация)	Чтение с локального диска	YAML / .env	Адреса, логины, токены, пути, расписание

По таблице 3 видно, что сетевые протоколы используются только на границе модуля интеграции с внешними системами (SFTP, HTTPS, TCP СУБД), а внутри модуля данные передаются вызовами функций. Все внешние каналы защищены: SFTP шифрует обмен по SSH, HTTPS — по TLS [2], а подлинность вебхуков проверяется подписью HMAC [3].

ЗАКЛЮЧЕНИЕ

В ходе практической работы для каждого модуля системы «Система агрегации и интеграции данных интернет-магазина» обоснован уровень интеграции: компоненты обмена файлами и записи в БД работают на уровне данных, компоненты приёма вебхуков и обмена с маркетплейсом — на уровне интерфейсов (API), сервисы валидации заказов и расчёта остатков — на уровне бизнес-логики.

Разработана модульная структура проекта из трёх модулей (источник/интерфейс, бизнес-логика, приёмник/хранилище) и вспомогательных компонентов, построена схема связей и пошагово описаны четыре потока данных. Составлена спецификация артефактов и протоколов: для одиннадцати связей указаны уровень, протокол, формат и назначение данных. Схема и спецификация размещены в каталоге `docs/pr2` репозитория и будут основой для реализации модулей в следующих работах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Техническое описание мини-проекта «Система агрегации и интеграции данных интернет-магазина» : приложение к практической работе № 1 по МДК 02.02 / СПбИЭУ. – Санкт-Петербург, 2026. – Текст : электронный // ЭИОС СПбИЭУ. – URL: https://spbiem.tech (дата обращения: 10.10.2026).

2. RFC 9110. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. – IETF, 2022. – URL: https://www.rfc-editor.org/rfc/rfc9110 (дата обращения: 10.10.2026). – Текст : электронный.

3. RFC 2104. HMAC: Keyed-Hashing for Message Authentication / H. Krawczyk, M. Bellare, R. Canetti. – IETF, 1997. – URL: https://www.rfc-editor.org/rfc/rfc2104 (дата обращения: 10.10.2026). – Текст : электронный.

4. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : межгосударственный стандарт : утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 26 декабря 1990 г. № 3294 : дата введения 1992-01-01. – URL: https://docs.cntd.ru/document/9041994 (дата обращения: 10.10.2026). – Текст : электронный.

5. Matplotlib : Visualization with Python : документация. – URL: https://matplotlib.org/stable/ (дата обращения: 10.10.2026). – Текст : электронный.